Informe Proyecto 3 — Pochoco SoC en GF180MCU

Integrantes: José Tomás Arévalo, Santiago Arenas y Danna Collazos.

Resumen

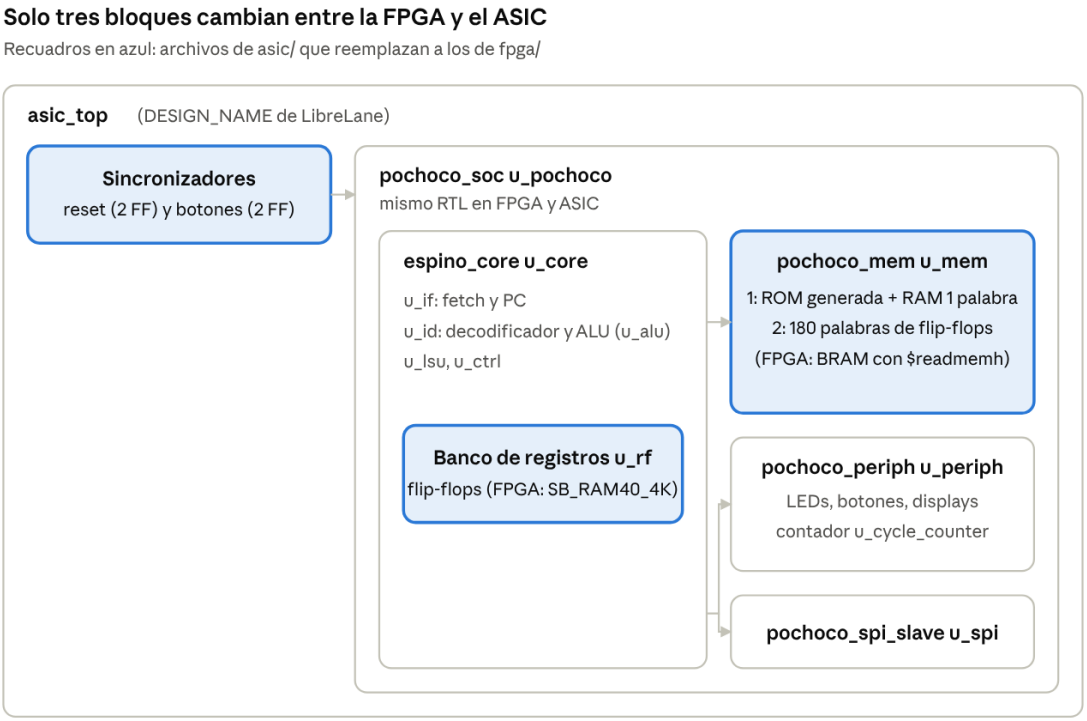
El Pochoco SoC con el juego de reflejos del Proyecto 2 se llevó de RTL a GDSII con LibreLane 3.0.17 y el PDK GF180MCU (`gf180mcuD`, biblioteca `gf180mcu_fd_sc_mcu7t5v0`), con `CLOCK_PERIOD` = 100 ns y `FP_CORE_UTIL` = 40 %. El resultado es un macro de celdas estándar con top `asic_top`.

- **Adaptación a ASIC.** El SoC quedó sin elementos propios de la FPGA. El banco de registros pasó de `SB_RAM40_4K` a flip-flops, el reset de encendido con valor inicial se reemplazó por un pin de reset sincronizado, y los botones pasan por un sincronizador de dos flip-flops.
- **Memoria.** Se implementaron las dos estrategias del enunciado: una ROM combinacional generada por el ensamblador más una RAM de 1 palabra, y una memoria unificada de 180 palabras de flip-flops cuyo valor de reset es el programa. El juego usa 179 palabras de instrucciones y 1 de datos.
- **Resultado de la corrida final.** *Pendiente de la corrida de LibreLane.*
- **Verificación.** El juego completo pasa 28/28 pruebas en el RTL del ASIC con las dos memorias. La simulación post-síntesis reproduce el RTL ciclo a ciclo.

Las cifras marcadas como *estimación local* salen de Yosys 0.70 + ABC con la biblioteca GF180 del mismo PDK, sin place & route. Las definitivas son las de las corridas de LibreLane, en `asic/resultados/`.

3.1 Del RTL de FPGA al RTL de ASIC

`asic/asic_top.v` instancia el mismo `pochoco_soc` que usa la FPGA. Todo lo propio de la iCE40 o de la Go Board se sacó del SoC: quedó en `fpga/` o se reemplazó en `asic/`. La memoria se instancia con un solo nombre, `pochoco_mem` (instancia `u_mem`), y cada flujo aporta su implementación.



jerarquía de asic_top · nombres de instancia del RTL

El núcleo, los periféricos y el SPI son el mismo RTL en las dos plataformas. Solo cambian tres bloques: la memoria (con el programa generado), el banco de registros y el top (reset y sincronizadores).

Puertos del macro

Son terminales lógicos del bloque, no pads.

Puerto	Dir.	Función
`clk`	entrada	Reloj del sistema, T = 100 ns (10 MHz)
`rst_n`	entrada	Reset asíncrono, activo en bajo
`btn_i[3:0]`	entrada	Botones; botón *i* = bit *i* de `0x8000_0008`
`led_o[3:0]`	salida	LEDs; LED *i* = bit *i* de `0x8000_0004`
`seg1_o[6:0]`, `seg2_o[6:0]`	salida	Displays (nibble alto y bajo de `0x8000_0000`), activos en alto, bit 0 = A … bit 6 = G
`spi_sclk_i`, `spi_mosi_i`, `spi_cs_n_i`, `spi_miso_o`	E/S	Esclavo SPI del Pochoco (sin uso en el juego)

Construcciones que dependían de la FPGA

Para obtener la evidencia, el RTL del Proyecto 2 (commit `45f33d8`, copiado sin cambios en `asic/evidencia_p2/rtl/`) se pasó por LibreLane en dos corridas: `make evidencia`. La columna de evidencia indica el log y la línea; *los números de línea de los logs se completan con la corrida*.

#	Construcción (Proyecto 2)	Evidencia en LibreLane	Reemplazo en el Proyecto 3
1	Banco de registros con `SB_RAM40_4K` (`espino_register_file.v:22` y `:33`)	Error del lint: `%Error-MODMISSING: ... Cannot find file containing module: 'SB_RAM40_4K'`. Log: `runs/p2_paso1/*-verilator-lint/verilator-lint.log`, línea *pendiente*. El flujo se detiene en `Checker.LintErrors`.	`asic/espino_register_file.v`: 15 registros de 32 bits con flip-flops (x0 no se implementa), dos puertos de lectura registrados y la misma latencia de 1 ciclo que la BRAM.
2	Reset de encendido con valor inicial: `reg [3:0] por_cnt = 4'b0` (`pochoco_soc.v:35`)	Warning del lint: `%Warning-PROCASSINIT: ... Procedural assignment to declaration with initial value: 'por_cnt'`. Log: `runs/p2_paso2/*-verilator-lint/verilator-lint.log`, línea *pendiente*. Yosys solo informa `Set init value: \por_cnt = 4'0000`.	Pin `rst_n` con sincronizador en `asic_top`. El contador de encendido quedó en `fpga/game_top.v`.
3	`$readmemh` dentro de un bloque `initial` (`pochoco_ram.v:35-36`)	**Ningún mensaje.** Yosys encuentra `sw/game.hex` (la ruta es relativa a la carpeta desde donde se corre LibreLane) y lo guarda como valor inicial de la memoria. Log: `runs/p2_paso2/*-yosys-synthesis/*.log`.	La ROM generada por el ensamblador (estrategia 1) o el programa como valor de reset de los flip-flops (estrategia 2). Ver 3.2.
4	Inferencia de BRAM: memoria de 512 × 32 que la iCE40 implementaba en `SB_RAM40_4K`	**Ningún mensaje.** Yosys la convierte en flip-flops: `Mapping memory \u_pochoco.u_ram.mem`. El diseño pasa de 0,14 a unos 2,38 mm² de celdas (*estimación local*).	Dos memorias dimensionadas según el uso real del juego (3.2).
5	Botones conectados sin sincronizar al registro de lectura del periférico	**Ningún mensaje.**	Sincronizador de dos flip-flops por botón en `asic_top`.
6	Segmentos activos en bajo (display de ánodo común de la placa),	**Ningún mensaje.**	El SoC entrega los segmentos activos en alto. La inversión quedó en

#	Construcción (Proyecto 2)	Evidencia en LibreLane	Reemplazo en el Proyecto 3
	invertidos dentro del SoC		`fpga/game_top.v`.
7	Reloj de 25 MHz supuesto en `game.s` (`FRECUENCIA`)	**Ningún mensaje.**	`-D FRECUENCIA=10000000` al generar la ROM (ver 3.4).

Por qué eran un problema aunque el flujo no avisara

- **Valores iniciales (filas 2 y 3).** En la iCE40 todos los flip-flops y las BRAM arrancan con el valor del bitstream. En silicio un flip-flop arranca en un valor indefinido, y las celdas de GF180 no tienen "valor inicial". Yosys acepta el atributo `init`, pero no hay celda que lo implemente: el chip encendería sin programa y sin reset.
- **Memoria (fila 4).** Sin BRAM, la memoria de 512 palabras se convierte en 16.384 flip-flops con sus multiplexores. Es correcta, pero ocupa casi 17 veces el resto del diseño.
- **Botones (fila 5).** Una señal asíncrona que llega a un flip-flop cerca del flanco del reloj puede dejarlo metaestable. En la FPGA el riesgo existía igual; en un ASIC no se puede corregir después de fabricar. El esclavo SPI ya tenía sincronizadores.
- **Reset (fila 2).** Además de llegar por un pin, el reset se suelta en sincronía con el reloj (activación asíncrona, liberación síncrona con dos flip-flops). Así todos los flip-flops salen del reset en el mismo ciclo, y ninguno ve el flanco de reset cerca del de reloj (*recovery/removal*).

3.2 Memoria y programa del juego

La ROM con RAM reducida ocupa unas **7 veces menos área** que la memoria unificada de flip-flops (0,14 contra 0,97 mm² de celdas, *estimación local*), con la misma frecuencia. Por eso es la estrategia de la corrida final.

Las dos estrategias implementan el mismo módulo `pochoco_mem`, con la misma interfaz y la misma latencia que la BRAM de la FPGA: el dato sale registrado un ciclo después de la dirección. El Espino no cambia, y los dos diseños ejecutan el juego en los mismos ciclos. Se elige una u otra según el archivo que lista la configuración: `asic/config.json` o `asic/config_ff.json`.

Cuánto usa el juego

	Palabras	Cómo se obtuvo
Instrucciones	**179** (`0x000` a `0x2C8`)	El ensamblador cuenta las palabras hasta la etiqueta `fin_programa` (en 0x2CC). Lo informa al generar la ROM: `179 de 256 palabras`.
Datos	**1** (la pila)	Análisis del programa: la única subrutina que usa la pila es `soltar_botones`, que guarda `ra` porque llama a `esperar`; `esperar` no llama a nadie. Confirmado en simulación: en una partida completa hay 28 escrituras y

	Palabras	Cómo se obtuvo
		28 lecturas de datos, todas en la misma dirección.

Ambas cifras están dentro del límite recomendado (256 de instrucciones y 64 de datos).

Para que el mismo programa sirviera con una ROM separada se hicieron dos cambios en `game.s`:

1. `elegir_objetivo` leía la máscara del LED de una tabla `.word 1, 2, 4, 8`. Con la ROM, el puerto de datos ya no ve el programa, así que la máscara `1 << índice` se calcula sumando (`a0 = a0 + a0`, índice veces).
2. La pila parte justo después del programa: `li sp, fin_programa + 4 * PILA_PALABRAS`, con `.equ PILA_PALABRAS, 1`. Así el tamaño de la memoria unificada sale del propio programa.

Extensión del ensamblador

`asm/asm.py` tiene dos opciones nuevas que generan el programa como hardware:

Opción	Genera	Contenido
`--rom game_rom.v`	`module game_rom`	Un `case` sobre la dirección de palabra (8 bits), con una palabra de 32 bits por dirección y el comentario de la instrucción de origen; `default` = 0. Falla si el programa pasa de 256 palabras.
`--ffmem game_ffmem.vh`	`GAME_WORDS`, `GAME_IMAGE`	La imagen completa (programa + `PILA_PALABRAS`) como un vector de 32 × N bits, para usarlo como valor de reset.

Un mismo comando genera las dos desde `sw/game.s`, con las constantes redefinidas por `-D` (`bash asic/scripts/generar.sh`).

Estrategia 1: ROM combinacional + RAM reducida (`asic/pochoco_mem_rom.v`)

- **Puerto de instrucciones:** la ROM generada es lógica combinacional pura. Yosys la reduce como cualquier otra función de 8 entradas y 32 salidas, sin flip-flops. La salida se registra en `pochoco_mem` (32 flip-flops) para mantener la latencia.
- **Puerto de datos:** una RAM de `DATA_WORDS` = 1 palabra (32 flip-flops con byte enables), igual a lo que usa el juego. La decodificación es parcial: cualquier dirección que no sea de periférico cae en esa palabra. Por eso el programa no puede leer datos de la ROM.

Estrategia 2: memoria unificada de flip-flops (`asic/pochoco_mem_ff.v`)

- 180 palabras = 179 de programa + 1 de pila, de 32 flip-flops cada una, ambas generadas por el ensamblador.

- En silicio no hay `$readmemh`: el programa es el **valor de reset**. Cada bit en 0 usa un flip-flop con reset (`dffrnq`) y cada bit en 1, uno con set (`dffsnq`). Al soltar el reset la memoria ya contiene el juego.
- Los dos puertos leen cualquier palabra con un multiplexor de 180 a 1 por puerto, y el de datos también escribe.

Comparación

	ROM + RAM	Flip-flops
Palabras (instrucciones + datos)	179 + 1	180 unificadas
Área de celdas, síntesis (*estimación local*)	137.830 µm²	970.219 µm²
Flip-flops (*estimación local*)	847	6.575
Celdas (*estimación local*)	4.426	24.447
Retardo de la ruta crítica, corner lento (*estimación local, ABC sin cables*)	19,3 ns	20,1 ns
Área de celdas, LibreLane	*pendiente*	*pendiente*
Flip-flops, LibreLane	*pendiente*	*pendiente*
Utilización final	*pendiente*	*pendiente*
Frecuencia máxima (menor período que cierra)	*pendiente*	*pendiente*

La diferencia de área viene de los 5.760 flip-flops de la memoria unificada: un flip-flop con set o reset ocupa 75 a 79 µm², frente a unos 11 µm² de una NAND2. La ROM guarda los mismos 5.728 bits como lógica: la función del `case` se minimiza y no necesita un flip-flop por bit. La frecuencia casi no cambia porque la ruta crítica no pasa por la memoria (ver 3.4). En el layout, los multiplexores de 180 a 1 y el área siete veces mayor agregan cables largos, lo que la corrida de LibreLane debería reflejar.

3.3 Síntesis y trazabilidad jerárquica

El banco de registros es el módulo más grande, con **la mitad del área**. El contador de ciclos agregado en el Proyecto 2 aporta un **2,7 %** (*estimación local; la cifra final sale de `stat.rpt` de la corrida*).

Método de trazabilidad

LibreLane aplana la jerarquía por defecto. Para conservar los tres bloques pedidos se usa `SYNTH_KEEP_HIERARCHY_MODULES = [espino_register_file, espino_alu, pochoco_cycle_counter]` en `asic/config.json`. El resto se aplana y se optimiza normalmente.

- **En la síntesis**, `stat.rpt` reporta el área de cada módulo conservado por separado. El área propia del top es el total menos la de esos tres. Lo calcula `python3 asic/scripts/resumen.py --corrida final`.
- **En el layout**, OpenROAD aplana al leer el netlist, y cada celda queda con el nombre de su instancia como prefijo: `u_pochoco.u_core.u_rf/…`, `u_pochoco.u_core.u_id.u_alu/…` y

`u_pochoco.u_periph.u_cycle_counter/…`. `asic/scripts/resaltar.tcl` suma el área de las celdas con cada prefijo y las marca en la GUI (ver 3.6).

- El contador de ciclos era un registro dentro del periférico. Para poder rastrearlo se separó en su propio módulo (`rtl/pochoco_cycle_counter.v`), sin cambiar su lógica.

Área por módulo

Módulo	Instancia	Área local (µm²)	%	Área LibreLane (µm²)
Banco de registros	`u_pochoco.u_core.u_rf`	69.469	51,1 %	*pendiente*
ALU	`u_pochoco.u_core.u_id.u_alu`	8.970	6,6 %	*pendiente*
Contador de ciclos	`u_pochoco.u_periph.u_cycle_counter`	3.716	2,7 %	*pendiente*
Resto (aplanado): ROM, decodificador, fetch, LSU, periféricos, SPI	`asic_top`	53.725	39,5 %	*pendiente*
Total		**135.881**	100 %	*pendiente*

La suma de esta tabla (135.881 µm²) y el total de la comparación de 3.2 (137.830 µm²) difieren en 1.949 µm² (1,4 %). *Falta confirmar con el stat.rpt de la corrida final de dónde sale la diferencia; si viene de conservar la jerarquía de los tres módulos, se anota aquí.*

Las cinco celdas más usadas

Celda	Cantidad local	Qué parte del diseño las explica
`mux2_1`	718	La biblioteca no tiene flip-flop con *enable*: cada flip-flop que solo se escribe a veces lleva un `mux2` que recircula su valor. Son 480 del banco de registros, más el PC, el registro de instrucción y los registros de periféricos.
`nand2_1`	552	Los dos multiplexores de lectura del banco de registros (15 a 1, 32 bits cada uno), la lógica de la ROM y el decodificador.
`dffq_1`	512	Flip-flops sin reset: 480 de los 15 registros de 32 bits, más los 32 de la palabra de RAM.
`dffrnq_1`	331	Flip-flops con reset asíncrono: PC (2 × 32), contador de ciclos (32), salidas registradas de memoria y banco de registros (4 × 32), periféricos, SPI y sincronizadores.
`aoi21_1`	282	Compuertas AND-OR-invertidas de la ROM y de los multiplexores.

Cantidades de la síntesis local; las de la corrida final las imprime `resumen.py`.

Lo esperado y lo obtenido

Bloque	Estimación antes de sintetizar	Resultado local	Diferencia
Contador de ciclos	32 flip-flops con reset (32 × 74,6 µm² ≈ 2.390 µm²) + incrementador de 32 bits (≈ 1.300 µm²) ≈ 3.700 µm²	3.716 µm²	Coincide.
Banco de registros	480 flip-flops (≈ 30.600 µm²) + 480 `mux2` de escritura (≈ 13.700 µm²) ≈ 45.000 µm²	69.469 µm²	Un 50 % más: la estimación olvidó los **dos multiplexores de lectura** de 15 a 1 × 32 bits y los 64 flip-flops de salida. Con la BRAM de la FPGA, esa lectura la hacía el bloque de memoria.
ALU	Sumador/restador de 32 bits, comparadores y lógica, sin shifts	8.970 µm²	Coincide en orden de magnitud. Los shifts siguen desactivados como en el Proyecto 2.

El contador agrega 32 flip-flops a los 847 del diseño (3,8 %) y un 2,7 % del área: en el ASIC es barato. En la FPGA del Proyecto 2 pesaba más (de 84 % a 91 % de las celdas lógicas) porque la iCE40 casi no tenía espacio libre.

3.4 Timing

Con T = 100 ns el diseño tiene mucho margen: la lógica más lenta tarda unos 19 ns en el corner lento (*estimación local*). El período mínimo que cierra en todos los corners sale del barrido de LibreLane: *pendiente*.

Barrido de `CLOCK_PERIOD`

Corners de GF180 en LibreLane: proceso tt, ss y ff × parásitos min, nom y max (9 en total). El típico es `nom_tt_025C_5v00` y el más lento, `max_ss_125C_4v50` (proceso lento, 125 °C, 4,5 V, cables con resistencia y capacidad máximas). Una corrida cierra timing si WNS ≥ 0 y TNS = 0 en setup y hold en todos los corners. Comando: `make barrido-reloj`.

`CLOCK_PERIOD` (ns)	Setup WNS / TNS típico (ns)	Hold WNS / TNS típico (ns)	Setup WNS / TNS lento (ns)	Hold WNS / TNS lento (ns)	Área de celdas (µm²)	¿Cierra?
100	*pendiente*					

CLOCK_PERIOD (ns)	Setup WNS / TNS típico (ns)	Hold WNS / TNS típico (ns)	Setup WNS / TNS lento (ns)	Hold WNS / TNS lento (ns)	Área de celdas (µm²)	¿Cierra?
50	*pendiente*					
35	*pendiente*					
30	*pendiente*					
25	*pendiente*					
20	*pendiente*					

Cada `CLOCK_PERIOD` cambia las restricciones, y con ellas lo que hacen la síntesis (ABC usa el período como objetivo) y el *resizer* de OpenROAD: buffers, tamaño de celdas y placement. Por eso el área de celdas también se reporta por período, y debe crecer al acercarse al límite. La frecuencia máxima se informa como 1 / (menor período que cierra en todos los corners): *pendiente*.

Ruta crítica

En la estimación local, la ruta más lenta va del registro de instrucción a la escritura en el banco de registros. *La celda de inicio y la de fin exactas se toman de `max.rpt` del corner lento de la corrida final; `resumen.py --corrida final` las imprime.*

	Estimación local	Corrida final
Inicio	Flip-flop de `u_pochoco.u_mem.a_rdata_o` (la instrucción leída)	*pendiente*
Recorrido	decodificador (`u_decoder`) → selección de operandos → ALU (`u_alu`, suma o comparación de 32 bits) → multiplexor de `rf_wdata`	*pendiente*
Fin	Multiplexor de escritura de un flip-flop de `u_rf.regs`	*pendiente*
Módulo	`espino_id_stage` + `espino_register_file`	*pendiente*
Función de Espino	**Decode, ejecución y write-back en el mismo ciclo**: Espino ejecuta y escribe el resultado en la etapa ID	*pendiente*

Cambio de RTL propuesto: predecodificar

Cada instrucción de Espino ya pasa un ciclo esperando la lectura síncrona del banco de registros (`ops_valid_q = 0`, `stall` activo). Durante ese ciclo el fetch está detenido y la instrucción no cambia. La propuesta es **registrar las salidas del decodificador en ese ciclo** (`dec_q`, 82 flip-flops). En el ciclo de ejecución la instrucción ya está decodificada, y el decodificador sale de la ruta crítica.

- **No cambia ningún ciclo.** El juego completo pasa 28/28 y la simulación post-síntesis da exactamente los mismos eventos, ciclo a ciclo, que sin el cambio.
- Se activa con `VERILOG_DEFINES = ["ESPINO_PREDECODIFICAR"]` (está en `rtl/espino_core/espino_id_stage.v` dentro de un `ifdef`), así la corrida base y la optimizada usan el mismo código.

	Base	Predecodificado	Efecto
Retardo de la lógica, corner típico (*local*)	10,1 ns	9,4 ns	−7 %
Retardo de la lógica, corner lento (*local*)	19,3 ns	16,9 ns	−13 %
Área de celdas (*local*)	152.457 µm²	155.642 µm²	+2,1 %
Slack con LibreLane al período mínimo	*pendiente*	*pendiente*	
Área con LibreLane	*pendiente*	*pendiente*	

Las cifras locales de esta tabla vienen de ABC con buffering y dimensionamiento, por eso el área es mayor que la de 3.3. Con el cambio, la ruta crítica pasa a empezar en `dec_q` o en el PC: el costo de la ALU y del write-back sigue ahí.

Constantes de `game.s` para T = 100 ns

Con T = 100 ns el reloj es de 10 MHz y un ciclo dura 100 ns. Basta cambiar una constante: `FRECUENCIA` = 10.000.000 (antes 25.000.000). Las demás se calculan solas, porque `game.s` define todos los tiempos en décimas:

Constante	Cálculo	Valor	Antes (25 MHz)
`CICLOS_DECIMA`	10.000.000 / 10	1.000.000	2.500.000
`CICLOS_INICIO` (3 s)	30 × 1.000.000	30.000.000	75.000.000
`CICLOS_ERROR` (1 s)	10 × 1.000.000	10.000.000	25.000.000
`CICLOS_FINAL` (2 s)	20 × 1.000.000	20.000.000	50.000.000
`CICLOS_PARPADEO` (0,5 s)	5 × 1.000.000	5.000.000	12.500.000
`CICLOS_TOPE` (9,9 s)	99 × 1.000.000	99.000.000	247.500.000
`CICLOS_REBOTE` (20 ms)	1.000.000 / 5	200.000	500.000

Se regenera con `python3 asm/asm.py sw/game.s -o build/game_asic.hex -D FRECUENCIA=10000000 --rom asic/gen/game_rom.v --ffmem asic/gen/game_ffmem.vh`. Por ejemplo, `li a0, CICLOS_INICIO` (30.000.000 = `0x01C9C380`) queda como `lui a0, 0x01C9C` (`01c9c537`) + `addi a0, a0, 896` (`38050513`). El tiempo se sigue mostrando en décimas porque `ciclos_a_decimas` divide por `CICLOS_DECIMA`. El contador de 32 bits da la vuelta cada 2^{32} × 100 ns = 429,5 s, más que en la FPGA, y la resta sin signo sigue siendo correcta.

3.5 Floorplan, alimentación, placement y costo

El macro con ROM debería medir del orden de 0,4 mm² a 40 % de utilización, unos **US$ 9.000** con el supuesto lineal. La memoria de flip-flops lo multiplicaría por siete. *El área mínima que cierra y el costo final salen del barrido de utilización.*

Barrido de utilización

Con `FP_SIZING = relative`, LibreLane fija el área del core en (área de celdas) / `FP_CORE_UTIL`, con relación de aspecto 1. A mayor utilización el die es más chico, pero queda menos espacio para buffers, para el árbol de reloj y para rutear. Comando: `make barrido-util` (40, 55, 70 y 85 %).

`FP_CORE_UTIL` (%)	Área del die (µm²)	Utilización final	Errores de DRC del ruteo	Antenas	WNS setup lento (ns)	¿Completa el flujo con los criterios?
40	*pendiente*					
55	*pendiente*					
70	*pendiente*					
85	*pendiente*					

Gráfico de utilización contra violaciones (errores de DRC del ruteo y antenas): se agrega con los datos de `asic/resultados/corridas.csv`.

Si 85 % todavía cerrara, se probaría un die cuadrado de lado fijo más chico con `bash asic/scripts/barrido.sh die <lado>` hasta encontrar el límite.

Terminales del macro

`asic/pin_order.cfg` agrupa los terminales por función, en el lado donde estaría lo que se conecta a ellos:

Lado	Terminales	Por qué
Oeste	`btn_i[3:0]`	Entradas del usuario, juntas y en orden. Del otro lado están los botones del tablero.
Este	`led_o[3:0]`, `seg1_o[6:0]`, `seg2_o[6:0]`	Todas las salidas visuales, en el lado opuesto a las entradas. Las señales cruzan el bloque en la dirección del flujo de datos (entrada → SoC → salida) y no se cruzan entre sí. Cada bus queda contiguo.
Sur, al centro	`clk`, `rst_n`	Señales globales. El reloj entra por el centro de un lado para que el árbol de reloj (CTS) parta cerca del centro del core, con ramas parecidas hacia todos los

Lado	Terminales	Por qué
		flip-flops. El reset va al lado.
Norte	`spi_sclk_i`, `spi_mosi_i`, `spi_cs_n_i`, `spi_miso_o`	Interfaz SPI completa en un lado, como iría a un conector o a otro chip. Separada de las señales del juego.

Terminales del macro frente a pads de un chip completo

Un terminal del macro es solo un rectángulo de metal en el borde del bloque, que maneja la carga de un cable interno. En un chip completo, cada señal pasaría por un **pad** del anillo de I/O (biblioteca `gf180mcu_fd_io` del mismo PDK):

Señal	Pad que necesitaría	Por qué
`btn_i`, `rst_n`	Entrada con Schmitt trigger	Botones con transiciones lentas y ruido; protección ESD.
`clk`	Entrada	Protección ESD y un buffer que maneje el árbol de reloj.
`led_o`, `seg*_o`	Salida con capacidad de corriente	Un LED o segmento consume del orden de mA, que una celda estándar no puede entregar.
SPI	Entradas y una salida	Señales externas: ESD y niveles de voltaje.
Alimentación	Pads de VDD/VSS del core y de DVDD/DVSS del anillo	Varios pares, para repartir la corriente.

Son 28 pads de señal más los de alimentación. Cada pad mide del orden de 75 µm × 350 µm, así que un chip con este macro estaría limitado por los pads: el anillo ocuparía más que el propio diseño.

Red de alimentación

La genera LibreLane (`OpenROAD.GeneratePDN`) con los valores del PDK: anillos de VDD/VSS alrededor del core, y bandas verticales y horizontales sobre las filas de celdas. *Pendiente: capturas y reporte de caída IR (`ir__drop__worst`) de la corrida final.*

Costo estimado

Con US$ 23.000 por mm² y suponiendo, solo para este ejercicio, que el costo escala linealmente con el área:

Diseño	Área del die	Costo
ROM + RAM, 40 % (*estimación local*)	≈ 0,35 a 0,40 mm²	≈ US$ 8.000 a 9.200
Flip-flops, 40 % (*estimación local*)	≈ 2,4 mm²	≈ US$ 56.000
ROM + RAM, corrida final	*pendiente*	*pendiente*

Diseño	Área del die	Costo
Área mínima que cierra (barrido)	*pendiente*	*pendiente*

Limitaciones de la aproximación lineal:

- **Costos fijos.** El juego de máscaras, el diseño y la verificación se pagan una vez, sin importar el área. En lotes chicos dominan el costo.
- **Área mínima y bloques fijos.** Los servicios multiproyecto (MPW) venden área en bloques. Un diseño de 0,4 mm² paga igual el mínimo.
- **Pads y anillo de sellado.** No están en el macro y, como se vio arriba, ocuparían más que el núcleo.
- **Rendimiento (yield).** La probabilidad de defectos crece con el área de forma no lineal, así que un die el doble de grande cuesta más del doble por chip bueno.
- **Encapsulado, prueba y volumen.** Se suman por chip y bajan con la cantidad.

Decisión con más impacto: la estrategia de memoria. Pasar de flip-flops a ROM baja el área unas 7 veces, de ≈ US$ 56.000 a ≈ US$ 9.000. Después viene la utilización: pasar de 40 % a 70 % reduciría el die en un 40 % adicional, si sigue cerrando. El período de reloj casi no influye a 100 ns, porque sobra margen y el *resizer* no necesita agrandar celdas.

3.6 Signoff y layout anotado

Resultado pendiente de la corrida final. El criterio de cierre es DRC limpio (Magic y KLayout), LVS limpio (Netgen) y ninguna violación de antena sin justificar.

Verificación	Herramienta	Métrica	Resultado
DRC	Magic	`magic__drc_error__count`	*pendiente*
DRC	KLayout	`klayout__drc_error__count`	*pendiente*
LVS	Netgen	`design__lvs_error__count`	*pendiente*
Antena	OpenROAD	`antenna__violating__nets` / `__pins`	*pendiente*
Ruteo	OpenROAD	`route__drc_errors`	*pendiente*

Los reportes quedan en `asic/resultados/final/` (`drc_magic.rpt`, `drc_klayout.json`, `lvs.rpt`, `antena.rpt`), copiados por `make recolectar`.

Antenas

LibreLane repara las antenas en dos pasos: inserta diodos en las redes con riesgo después del ruteo global (`OpenROAD.RepairAntennas`) y vuelve a revisar después del ruteo detallado (`OpenROAD.CheckAntennas`). *Si queda alguna violación se justificará aquí con el reporte: red, capa, razón entre área de metal y de compuerta, y por qué no afecta.*

Violaciones durante el desarrollo

Pendiente de la corrida final. Aquí se describe una violación que haya aparecido en las iteraciones (por ejemplo, antena, DRC del ruteo o timing), con la captura del layout que marca la red o la zona afectada y el cambio que la resolvió. Si en ninguna corrida apareció alguna, se dejará dicho, con el run y el reporte que lo respaldan.

Register file, ALU y contador en el layout

Se identifican por el prefijo jerárquico que conservan las celdas (ver 3.3):

1. `librelane --pdk gf180mcuD --run-tag final --flow OpenInOpenROAD asic/config.json`
2. En la consola Tcl de OpenROAD: `source asic/scripts/resaltar.tcl`
3. El script marca cada bloque con un color (grupo 0 = register file, 1 = ALU, 2 = contador), imprime cuántas celdas y cuánta área tiene cada uno después del place & route, y guarda `asic/resultados/layout_bloques.png`.

Bloque	Celdas en el layout	Área (µm²)	% del total
Register file (`*u_rf/*`)	*pendiente*		
ALU (`*u_alu/*`)	*pendiente*		
Contador de ciclos (`*u_cycle_counter/*`)	*pendiente*		

Captura anotada pendiente.

3.7 Simulación funcional post-síntesis

El netlist post-síntesis reproduce el RTL **ciclo a ciclo**: con los mismos estímulos, los 13 cambios de LEDs y display ocurren en los mismos ciclos en las dos simulaciones. El programa mide **19 ciclos** en la ronda 1 y **33** en la ronda 2, y la pulsación incorrecta reinicia la ronda sin contarla. *Verificado con un netlist GF180 sintetizado localmente; falta repetirlo con el netlist de la corrida de LibreLane (`make postsint`).*

Configuración acortada

Las dos simulaciones usan el mismo programa, generado con `-D FRECUENCIA=10000000 -D CICLOS_DECIMA=1 -D RONDAS=2` (carpeta `asic/gen_postsint/`):

- `CICLOS_DECIMA = 1`: una "décima" dura un ciclo. La fase inicial dura 30 ciclos en vez de 3 s, y **el display muestra directamente los ciclos que midió el programa**, porque décimas = ciclos / 1.
- `RONDAS = 2`: la partida termina tras dos rondas correctas. Si el error contara como ronda, terminaría antes.

LibreLane sintetiza ese programa con `asic/config_postsint.json` y se detiene después de Yosys (`--to Yosys.Synthesis`). El netlist se simula con los modelos Verilog de las celdas GF180 del PDK, sin retardos (`-DFUNCTIONAL`). Todo lo hace `make postsint`.

Estímulos

`tb/tb_postsintesis.v` aprieta los botones en ciclos fijos, contados desde que se suelta el reset, sin mirar los LEDs. Cada pulsación dura 40 ciclos, más que una vuelta del bucle que lee los botones (14 ciclos). Los estímulos cambian en el flanco de bajada del reloj, para que el RTL y el netlist no resuelvan distinto una carrera del testbench.

Ciclo	Botón	Qué representa
20	`0001`	Empezar la partida
340	`0001`	Ronda 1, botón correcto (el LED se enciende en el ciclo 327)
900	`0100`	Ronda 2, botón **equivocado** (el LED encendido es `0001`)
1290	`1000`	Ronda 2 repetida, botón correcto (LED en el ciclo 1262)

Resultado

Así quedó el log de eventos. Fue idéntico en RTL y netlist, y también con la memoria de flip-flops:

Ciclo	LEDs	Display	Qué pasa
0	0000	00	Pantalla de inicio
110	1111	00	Fase inicial de la ronda 1
327	0001	00	LED objetivo encendido: empieza la medición
611	0001	**19**	Ronda 1 correcta: el programa midió 19 ciclos
672	1111	19	Fase inicial de la ronda 2
889	0001	19	LED objetivo de la ronda 2
934	0001	**EE**	Botón equivocado
936	0000	EE	LEDs apagados
1018	1111	EE	**La ronda 2 se reinicia**: vuelve la fase inicial (30 ciclos o más)
1262	1000	EE	Nuevo LED objetivo de la misma ronda
1704	1000	**33**	Ronda 2 correcta: 33 ciclos medidos
1765	0000	33	Fin de la partida tras 2 rondas correctas
2109	0000	**26**	Promedio (19 + 33) / 2 = 26: el error no se contó

El testbench verifica 10 condiciones (10/10 en RTL y en netlist), y el script compara los dos logs con `diff`:

- Una ronda correcta mide los mismos ciclos en RTL y netlist: 19 y 33.

- Una pulsación incorrecta muestra EE, apaga los LEDs y reinicia la ronda actual: vuelve la fase inicial, sale otro objetivo y la partida sigue necesitando 2 rondas correctas.
- La fase reiniciada dura al menos sus 30 ciclos.

El juego completo, con la espera acortada a una décima de 2.500 ciclos, también se verificó en el RTL del ASIC: 28/28 pruebas con cada memoria (`make sim-asic`).

Reproducibilidad

Todo se reproduce desde el repositorio con LibreLane 3.0.17 y el PDK que esa versión fija. Las versiones exactas de cada herramienta quedan en `asic/resultados/versiones.txt` (`make versiones`).

	Versión
LibreLane	3.0.17 (tag `3.0.17`, commit `64ef808`)
PDK	`gf180mcuD`, open_pdks `54435919abffb937387ec956209f9cf5fd2dfbee`
Biblioteca de celdas	`gf180mcu_fd_sc_mcu7t5v0` (7 tracks, 5 V)
Yosys, OpenROAD, Magic, Netgen, KLayout	*las del entorno Nix de LibreLane 3.0.17; pendiente copiar de `versiones.txt`*

Qué	Comando (desde la raíz del repositorio, en `nix-shell`)
Regenerar el programa	`make gen`
Corrida final	`make asic`
Estrategia 2	`make asic-ff`
Evidencia de la sección 3.1	`make evidencia`
Barridos	`make barrido-reloj`, `make barrido-util`
Predecodificado	`librelane --pdk gf180mcuD --run-tag predecodificado -c 'VERILOG_DEFINES=["ESPINO_PREDECODIFICAR"]' -c CLOCK_PERIOD=<T> asic/config.json`
Simulaciones	`make sim-asic`, `make postsint`
Tablas	`make resumen`, `python3 asic/scripts/resumen.py --corrida final`
Layout marcado	`librelane --pdk gf180mcuD --run-tag final --flow OpenInOpenROAD asic/config.json` y `source asic/scripts/resaltar.tcl`

Referencias

- Documentación de LibreLane: tutorial, instalación con Nix, PDK, cierre de timing y archivo de orden de pines.
- PDK GF180MCU
- OpenROAD: comandos de la GUI (`select`, `gui::save_image`).
- Pochoco SoC, Nicolás Villegas, Universidad de los Andes.